

## AL-2002 - Artificial Intelligence Lab

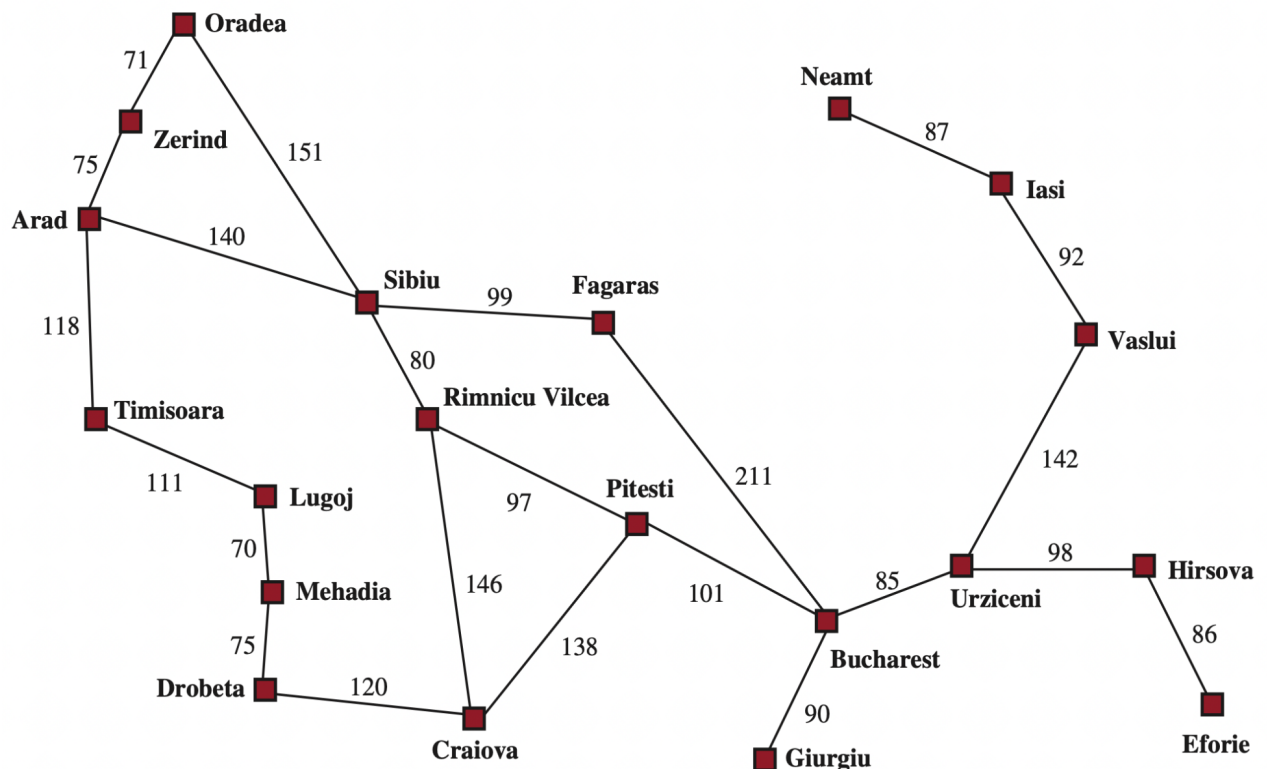
### Lab # 5

#### Objectives:

- Hill climbing + Local Beam Search + Simulated Annealing

**Note:** Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your program.
2. Write Program in multiple cells of the same notebook.
3. **Must Use Functions and Exception handling for each question, where they can be applied.**
4. **Do not copy from any source otherwise you will be penalized with negative marks.**
5. Add your source code in word document and attach output screenshots.
6. Upload word file and ipynb file.
7. Please submit your **Both files** with this naming convention `ROLLNO_SECTION_LABNO`. (**Do not upload zip file**).
8. Submit your lab on Google Classroom.



**Heuristic table: Euclidean distance to Bucharest (goal):**

|                    |             |
|--------------------|-------------|
| Arad 366           | Bucharest 0 |
| Craiova 160        | Dobreta 242 |
| Eforie 161         | Fagaras 178 |
| Giurgiu 77         | Hirsova 151 |
| Iasi 226           | Lugoj 244   |
| Mehadia 241        | Neamt 234   |
| Oradea 380         | Pitesti 98  |
| Rimnicu Vilcea 193 | Sibiu 253   |
| Timisoara 329      | Urziceni 80 |
| Vaslui 199         | Zerind 374  |

**Question 1:**

Using the provided weighted Romania road graph and your heuristic table of Euclidean distance to Bucharest (goal), run **hill climbing algorithm** from **Arad** to try to reach **Bucharest**: at each step, look only at the current city's immediate neighbors and move to the neighbor with the smallest heuristic value  $h(n)$  (if there is a tie, break it alphabetically); if no neighbor has a strictly smaller  $h$  than the current city, stop and report that you are stuck in a local minimum/plateau. Write the full sequence of visited cities (showing each city's  $h$  value), then compute the total path cost by summing the road distances along the edges you traverse.

Start: **Arad**

Goal: **Bucharest**

**Question 2:**

Using the provided **weighted Romania road graph** and the **heuristic table of Euclidean distance to Bucharest (goal)**, run the **Local Beam Search algorithm** with beam width  $B = 2$  to try to reach **Bucharest**.

At each step:

1. Start with the **initial beam containing the start city Arad**.
2. Generate **all immediate neighbors** of every city currently in the beam.
3. From all generated neighbors, **select the two cities with the smallest heuristic values  $h(n)$  to form the new beam**.
4. If there is a **tie**, break it **alphabetically**.
5. If **Bucharest is selected**, stop the search.
6. If **no newly generated neighbor has a strictly smaller heuristic value than the best city in the current beam**, stop and report that the search is **stuck in a local minimum or plateau**.

**Tasks:**

1. Write the **beam at each step**, showing the cities and their heuristic values  $h(n)$ .
2. Write the **final sequence of cities visited that leads to Bucharest (if reached)**.
3. Compute the **total path cost** by summing the road distances along the edges traversed.

**Start:** Arad  
**Goal:** Bucharest  
**Beam width:**  $B = 2$

**Question 3:**

```
SIMULATED_ANNEALING(initial_state):  
  
    current ← initial_state  
    best ← current  
    T ← initial_temperature  
  
    while T > stopping_temperature:  
  
        next ← RANDOM_NEIGHBOR(current)  
  
        ΔE ← COST(next) - COST(current)  
  
        if ΔE < 0 then  
            current ← next           // better solution  
        else  
            p ← random number between 0 and 1  
            if p < exp(-ΔE / T) then  
                current ← next       // accept worse solution with probability  
            end if  
        end if  
  
        if COST(current) < COST(best) then  
            best ← current  
        end if  
  
        T ← COOL(T)                  // reduce temperature  
  
    return best
```

Using the provided **weighted Romania road graph** and the **heuristic table of Euclidean distance to Bucharest (goal)**, run the **Simulated Annealing algorithm** to try to reach **Bucharest** starting from **Arad**.

At each step:

1. Start with the **initial state as the start city Arad**.
2. Set the **initial temperature**  $T = 100$ .
3. Select **one immediate neighbor** of the current city (consider neighbors in **alphabetical order**).
4. Compute the **change in heuristic value**

$$\Delta h = h(\text{neighbor}) - h(\text{current})$$

5. If the neighbor has a **smaller heuristic value**, move to that city.
6. If the neighbor has a **larger heuristic value**, accept the move with probability

$$p = e^{-\Delta h/T}$$

7. Reduce the temperature using the **cooling schedule**

$$T = 0.8 \times T$$

8. If **Bucharest is reached**, stop the search.

9. If the **temperature falls below**  $T < 1$ , stop and report that the search has terminated due to **low temperature**.

**Tasks:**

1. Write the **sequence of cities visited**, showing the **heuristic value**  $h(n)$  and the **temperature**  $T$  at each step.
2. Indicate whether each move was **accepted because it improved the heuristic** or **accepted probabilistically using the simulated annealing rule**.
3. Compute the **acceptance probability**  $P$  whenever a worse move is considered.
4. Compute the **total path cost** by summing the **road distances along the edges traversed**.

**Start:** Arad

**Goal:** Bucharest

**Initial Temperature:**  $T = 100$

**Cooling Schedule:**  $T = 0.8 \times T$

**Hint:**

import math

delta\_h = 76

T = 100

P = math.exp(-delta\_h / T)